

```

package main

import (
    "github.com/connerohnesorge/goffice/drawingml"
    "github.com/connerohnesorge/goffice/presentation/elements"
    "github.com/connerohnesorge/goffice/presentation/parts"
)

// chartSeriesConfig holds configuration for adding a chart series.
type chartSeriesConfig struct {
    seriesName      string
    categoryRange   string
    categoryValues []string
    valuesRange     string
    values          []float64
    seriesIndex     uint32
}

// scatterSeriesConfig holds configuration for adding a scatter chart series.
type scatterSeriesConfig struct {
    xRange      string
    xValues     []float64
    yRange      string
    yValues     []float64
    seriesIndex uint32
}

// addTitleToSlide adds a title shape to a slide with the given text.
func addTitleToSlide(
    slide *elements.Slide,
    titleText string,
) {
    titleShape := slide.AddShape()
    titleShape.SetPosition(titlePosX, titlePosY)
    titleShape.SetSize(titleWidth, titleHeight)
    titleTb := titleShape.GetOrCreateTextBody()
    titleP := titleTb.AddParagraph("")
    titleRun := titleP.AddRun(titleText)
    titleRun.SetFontSize(titleFontSize)
}

// createChartWithTitle creates a chart space with a title.
func createChartWithTitle(
    title string,
) (*drawingml.ChartSpace, *drawingml.Chart) {
    chartSpace := drawingml.NewChartSpace()
    chart := chartSpace.Chart()
    chartTitle := drawingml.NewTitleWithText(
        title,
    )
    chart.SetTitle(chartTitle)

    return chartSpace, chart
}

// axisResult holds the result of adding axes to a plot area.
type axisResult struct {
    xAxis *drawingml.ValueAxis
    yAxis *drawingml.ValueAxis
}

```

```

}

// addChartAxes adds category and value axes to a plot area.
func addChartAxes(
    plotArea *drawingml.PlotArea,
) (*drawingml.CategoryAxis, *drawingml.ValueAxis) {
    catAxis := plotArea.AddCategoryAxis(
        categoryAxisID,
        valueAxisID,
    )
    catAxis.SetAxisPosition(
        drawingml.AxisPositionBottom,
    )
    valAxis := plotArea.AddValueAxis(
        valueAxisID,
        categoryAxisID,
    )
    valAxis.SetAxisPosition(
        drawingml.AxisPositionLeft,
    )

    return catAxis, valAxis
}

// addValueAxes adds two value axes (for scatter charts) and returns them in a
// struct.
func addValueAxes(
    plotArea *drawingml.PlotArea,
) axisResult {
    xAxis := plotArea.AddValueAxis(
        categoryAxisID,
        valueAxisID,
    )
    xAxis.SetAxisPosition(
        drawingml.AxisPositionBottom,
    )
    yAxis := plotArea.AddValueAxis(
        valueAxisID,
        categoryAxisID,
    )
    yAxis.SetAxisPosition(
        drawingml.AxisPositionLeft,
    )

    return axisResult{xAxis: xAxis, yAxis: yAxis}
}

// Office default chart color palette (matches PowerPoint 2016+ colors).
var chartColors = []string{
    "4472C4", // Blue
    "ED7D31", // Orange
    "A5A5A5", // Gray
    "FFC000", // Yellow
    "5B9BD5", // Light Blue
    "70AD47", // Green
}

// addBarChartSeries adds a series to a bar chart using the provided configuration.
func addBarChartSeries(

```

```

    barChart *drawingml.BarChart,
    cfg *chartSeriesConfig,
) {
    series := barChart.AddSeries(
        cfg.seriesIndex,
        cfg.seriesIndex,
    )
    series.SetSeriesText(
        drawingml.NewSeriesTextWithValue(
            cfg.seriesName,
        ),
    )

    // Add shape properties with fill color for bar charts
    // This is required for LibreOffice to render the bars correctly
    spPr := drawingml.NewChartShapeProperties()
    colorIndex := cfg.seriesIndex % uint32(
        len(chartColors),
    )
    spPr.SetSolidFill(chartColors[colorIndex])
    series.SetShapeProperties(spPr)

    // Create category axis data with cached values
    cat := drawingml.NewCategoryAxisData()
    strRef := drawingml.NewStringReferenceWithCache(
        cfg.categoryRange,
        cfg.categoryValues,
    )
    cat.AppendChild(strRef)
    series.SetCategoryAxisData(cat)

    // Create values with cached numbers
    vals := drawingml.NewValues()
    numRef := drawingml.NewNumberReferenceWithCache(
        cfg.valuesRange,
        cfg.values,
    )
    vals.AppendChild(numRef)
    series.SetValues(vals)
}

// addLineChartSeries adds a series to a line chart using the provided
// configuration.
func addLineChartSeries(
    lineChart *drawingml.LineChart,
    cfg *chartSeriesConfig,
) {
    series := lineChart.AddSeries(
        cfg.seriesIndex,
        cfg.seriesIndex,
    )
    series.SetSeriesText(
        drawingml.NewSeriesTextWithValue(
            cfg.seriesName,
        ),
    )

    // Add shape properties with line stroke for line charts
    // Line charts need outline (stroke) properties, not fill

```

```

    spPr := drawingml.NewChartShapeProperties()
    colorIndex := cfg.seriesIndex % uint32(
        len(chartColors),
    )
    // Default line width in EMUs (28575 EMUs ≈ 1pt, matching Office defaults)
    const defaultLineWidth drawingml.EMU = 28575
    spPr.SetOutline(
        defaultLineWidth,
        chartColors[colorIndex],
    )
    series.SetShapeProperties(spPr)

    // Create category axis data with cached values
    cat := drawingml.NewCategoryAxisData()
    strRef := drawingml.NewStringReferenceWithCache(
        cfg.categoryRange,
        cfg.categoryValues,
    )
    cat.AppendChild(strRef)
    series.SetCategoryAxisData(cat)

    // Create values with cached numbers
    vals := drawingml.NewValues()
    numRef := drawingml.NewNumberReferenceWithCache(
        cfg.valuesRange,
        cfg.values,
    )
    vals.AppendChild(numRef)
    series.SetValues(vals)
}

// addAreaChartSeries adds a series to an area chart using the provided
// configuration.
func addAreaChartSeries(
    areaChart *drawingml.AreaChart,
    cfg *chartSeriesConfig,
) {
    series := areaChart.AddSeries(
        cfg.seriesIndex,
        cfg.seriesIndex,
    )

    // Add shape properties with fill color for area charts
    // This is required for LibreOffice to render the filled areas correctly
    spPr := drawingml.NewChartShapeProperties()
    colorIndex := cfg.seriesIndex % uint32(
        len(chartColors),
    )
    spPr.SetSolidFill(chartColors[colorIndex])
    // Manually append since AreaChartSeries doesn't have SetShapeProperties
    series.AppendChild(spPr)

    // Create category axis data with cached values
    cat := drawingml.NewCategoryAxisData()
    strRef := drawingml.NewStringReferenceWithCache(
        cfg.categoryRange,
        cfg.categoryValues,
    )
    cat.AppendChild(strRef)

```

```

series.SetCategoryAxisData(cat)

// Create values with cached numbers
vals := drawingml.NewValues()
numRef := drawingml.NewNumberReferenceWithCache(
    cfg.valuesRange,
    cfg.values,
)
vals.AppendChild(numRef)
series.SetValues(vals)
}

// addScatterChartSeries adds a series to a scatter chart using the provided
configuration.
func addScatterChartSeries(
    scatterChart *drawingml.ScatterChart,
    cfg *scatterSeriesConfig,
) {
    series := scatterChart.AddSeries(
        cfg.seriesIndex,
        cfg.seriesIndex,
    )

    // Add shape properties with fill color for scatter chart markers
    // This is required for LibreOffice to render the data points correctly
    spPr := drawingml.NewChartShapeProperties()
    colorIndex := cfg.seriesIndex % uint32(
        len(chartColors),
    )
    spPr.SetSolidFill(chartColors[colorIndex])
    // Manually append since ScatterChartSeries doesn't have SetShapeProperties
    series.AppendChild(spPr)

    // Create X values with cached numbers
    xVals := drawingml.NewXValues()
    xNumRef := drawingml.NewNumberReferenceWithCache(
        cfg.xRange,
        cfg.xValues,
    )
    xVals.AppendChild(xNumRef)
    series.SetXValues(xVals)

    // Create Y values with cached numbers
    yVals := drawingml.NewYValues()
    yNumRef := drawingml.NewNumberReferenceWithCache(
        cfg.yRange,
        cfg.yValues,
    )
    yVals.AppendChild(yNumRef)
    series.SetYValues(yVals)
}

// addDoughnutDataPointColors adds colored shape properties to each data point in a
doughnut chart series.
// Doughnut charts need individual data point formatting to display different
colors for each segment.
func addDoughnutDataPointColors(
    series *drawingml.PieChartSeries,
    categories []string,

```

```

) {
    for i := range categories {
        dPt := drawingml.NewDataPoint(uint32(i))
        spPr := drawingml.NewChartShapeProperties()
        colorIndex := i % len(chartColors)
        spPr.SetSolidFill(chartColors[colorIndex])
        dPt.SetShapeProperties(spPr)
        series.AppendChild(dPt)
    }
}

// addChartToSlide adds a chart to a slide by creating a ChartPart and linking it
// via a GraphicFrame.
func addChartToSlide(
    slidePart *parts.SlidePart,
    chartSpace *drawingml.ChartSpace,
) error {
    // Create a chart part in the slide
    chartPart, err := slidePart.AddChartPart()
    if err != nil {
        return err
    }

    // Replace the default chartSpace with the one we created
    chartPart.SetRootElement(chartSpace)

    // Get the slide and create a graphic frame to display the chart
    slide := slidePart.Slide()
    gf := slide.GetOrCreateShapeTree().
        AddGraphicFrame()

    // Position and size the graphic frame
    gf.SetPosition(chartPosX, chartPosY)
    gf.SetSize(chartWidth, chartHeight)

    // Link the graphic frame to the chart part
    elements.LinkGraphicFrameToChart(
        gf,
        chartPart.RelationshipID(),
    )

    return nil
}

```